

Using SNOMED CT with IRIS for Health and MCP

Using SNOMED CT with IRIS for Health and MCP

SNOMED CT terminology services built natively on InterSystems IRIS for Health 2026.2.0AI. Search, navigate, and explore 831K+ clinical concepts through SQL, ObjectScript, Python, or natural language via MCP tools for Claude Desktop and Claude Code.

What This Provides

- **RF2 Ingestion** — Load any SNOMED CT RF2 release directly into IRIS persistent storage
- **Lexical Search** — Full-text keyword search across all concept descriptions using iFind indexing
- **Semantic Search** — Find concepts by meaning using vector embeddings and cosine similarity
- **Hierarchy Navigation** — Traverse ancestors, descendants, parents, and children with O(1) subsumption checking
- **MCP Tools** — All functionality exposed as AI tools for Claude Desktop and Claude Code via the IRIS AI Hub framework

Prerequisites

Requirement	Purpose	Notes
InterSystems IRIS for Health 2026.2.0AI	Runtime platform	AI Hub EAP with %AI.Tool support
Docker	Container deployment	For the quickest setup
SNOMED CT RF2 Release	Terminology data	Requires license from your national release center
Ollama	Embedding generation	Optional — only needed for semantic search
iris-mcp-server	MCP gateway	Optional — only needed for Claude Desktop/Code integration

Getting SNOMED CT Data

SNOMED CT is a licensed terminology. Obtain the RF2 Snapshot release for your jurisdiction:

- **UK:** [NHS Digital TRUD](#) — SNOMED CT Monolith Edition, RF2: Snapshot (free under NHS England national licence)
- **International:** [MLDS](#) (allow up to a week for approval)
- **US:** [NLM UMLS](#)

Download the **Snapshot** (not Full or Delta) — it contains the current state of all concepts.

Getting Started

Option A: Docker (Recommended)

The fastest path from zero to a working system.

```
# 1. Clone and build
git clone <repository-url>
cd snomed-iris
docker compose up -d

# 2. Verify IRIS is running
docker compose ps
# Both iris and ollama services should show "healthy" / "running"
```

Option B: IPM Install (Existing IRIS Instance)

For teams already running IRIS for Health 2026.2.OAI:

```
// In the IRIS terminal (USER namespace):
zpm "install snomed-iris"
```

This compiles all classes, installs Python dependencies (requests), and registers the /mcp/snomed web application automatically.

Loading SNOMED CT Data

After installation, load your RF2 release files. The loader processes tab-separated RF2 files directly — no pre-processing needed.

Docker

```
# Copy the RF2 Snapshot directory into the container
docker cp
    /path/to/SnomedCT_InternationalRF2_PRODUCTION_20240101T120000Z/Snapshot/
    snomed-iris-iris-1:/tmp/snomed/

# Load concepts, descriptions, relationships, then compute hierarchy
docker exec -i snomed-iris-iris-1 iris session IRIS -U USER <<'EOF'
Do ##class(SNOMED.Loader).LoadConcepts("/tmp/snomed/Terminology/sct2_Concept_Snapshot_INT_20240101.t
Do ##class(SNOMED.Loader).LoadDescriptions("/tmp/snomed/Terminology/sct2_Description_Snapshot_INT_20
Do ##class(SNOMED.Loader).LoadRelationships("/tmp/snomed/Terminology/sct2_Relationship_Snapshot_INT_
Do ##class(SNOMED.TransitiveClosure).Compute()
Halt
EOF
```

Direct IRIS Terminal

```
// Load RF2 files (adjust paths to your RF2 location)
Do
##class(SNOMED.Loader).LoadConcepts("/data/snomed/sct2_Concept_Snapshot
Do
##class(SNOMED.Loader).LoadDescriptions("/data/snomed/sct2_Description_
Do
##class(SNOMED.Loader).LoadRelationships("/data/snomed/sct2_Relationsh:

// Compute transitive closure for hierarchy queries (takes ~30s for
831K concepts)
Do ##class(SNOMED.TransitiveClosure).Compute()
```

What Gets Loaded

RF2 File	IRIS Table	Record Count (UK Monolith)
sct2_Concept	SNOMED.Concept	~831K
sct2_Description	SNOMED.Description	~2.5M

RF2 File	IRIS Table	Record Count (UK Monolith)
sct2_Relationship (computed)	SNOMED.Relationship ^SNOMED.TC global	~1.8M O(1) subsumption lookup

Searching SNOMED CT

Lexical Search (Keyword)

Find concepts by matching keywords in their descriptions. Uses IRIS iFind full-text indexing for fast, linguistically-aware search.

ObjectScript:

```
Set results = ##class(SNOMED.Search).Lexical("diabetes mellitus", 10)
Write results
// Returns JSON array: [{"ConceptId": "73211009", "Term": "Diabetes mellitus", ...}, ...]
```

SQL:

```
SELECT d.ConceptId, d.Term
FROM SNOMED.Description d
WHERE d.%ID %FIND search_index(TermIdx, 'heart failure', 0, '*')
AND d.Active = 1
ORDER BY d.Term
```

Via MCP (natural language): > “Search SNOMED for concepts related to diabetes mellitus”

Semantic Search (By Meaning)

Find concepts whose embeddings are most similar to a natural language query. Requires Ollama with an embedding model.

Setting Up Embeddings

```
# Pull the embedding model (inside the Ollama container)
docker exec snomed-iris-ollama-1 ollama pull all-minilm

# Generate embeddings for all active concepts (~30 min for 831K)
docker exec -i snomed-iris-iris-1 iris session IRIS -U USER <<'EOF'
Do ##class(SNOMED.Embeddings).Generate("all-minilm", 100,
    "http://ollama:11434")
Halt
EOF
```

The generator is resumable — if interrupted, rerun the same command and it picks up where it left off.

Querying

ObjectScript:

```
Set results = ##class(SNOMED.Search).Semantic("chest pain when breathing", 10)
Write results
// Returns JSON: [{"ConceptId": "...", "Term": "...", "Score": 0.87}, ...]
```

Via MCP (natural language): > “Find SNOMED concepts semantically similar to difficulty breathing while lying down”

Results are ranked by cosine similarity (0–1). Only results above 0.5 threshold are returned.

Pattern Search (SQL LIKE)

Match descriptions using SQL wildcard patterns.

ObjectScript:

```
Set results = ##class(SNOMED.Search).Lexical("type 2 diab%", 10)
```

Via MCP: > “Search SNOMED for terms matching the pattern ‘chronickidney’”

Navigating the Hierarchy

SNOMED CT is organized as a polyhierarchy (concepts can have multiple parents). The transitive closure computed during loading enables instant ancestor/descendant queries.

Concept Lookup

ObjectScript:

```
Set json = ##class(SNOMED.Hierarchy).GetConcept("73211009")
Write json
// {"ConceptId": "73211009", "FSN": "Diabetes mellitus (disorder)",
"Active": 1, ...}
```

Via MCP: > “Look up SNOMED concept 73211009”

Parents and Children

ObjectScript:

```
// Immediate parents (IS-A targets)
Set parents = ##class(SNOMED.Hierarchy).GetParents("73211009")

// Immediate children
Set children = ##class(SNOMED.Hierarchy).GetChildren("73211009")
```

Via MCP: > “What are the children of Diabetes mellitus (73211009)?” > “What are the parent concepts of Type 2 diabetes?”

Ancestors and Descendants

ObjectScript:

```
// All ancestors up to the root
Set ancestors = ##class(SNOMED.Hierarchy).GetAncestors("44054006")

// All descendants (with limit)
Set descendants = ##class(SNOMED.Hierarchy).GetDescendants("73211009",
50)
```

Via MCP: > “Show me all ancestors of concept 44054006” > “Get the first 50 descendants of Diabetes mellitus”

Subsumption Checking (IS-A)

Check whether one concept is a subtype of another — O(1) performance via the precomputed transitive closure.

ObjectScript:

```
// Is Type 2 diabetes (44054006) a subtype of Diabetes mellitus
(73211009)?
Set result = ##class(SNOMED.Hierarchy).IsA("44054006", "73211009")
Write result
// {"result": true, "conceptId": "44054006", "ancestorId": "73211009"}
```

Via MCP: > “Is Type 2 diabetes mellitus a subtype of Diabetes mellitus?”

Using MCP Tools with Claude

The Model Context Protocol (MCP) allows Claude Desktop and Claude Code to call SNOMED tools directly during conversation. Nine tools are available:

Tool Reference

Tool	What It Does	Parameters
GetConcept	Look up a concept by its SNOMED CT identifier	conceptId (string)
LexicalSearch	Full-text keyword search across descriptions	searchTerm (string), maxResults (int, default 20)
SearchByPattern	SQL LIKE pattern matching on descriptions	pattern (string), maxResults (int, default 20)
SemanticSearch	Find concepts by meaning via vector similarity	query (string), maxResults (int, default 10)
GetParents	Get immediate parent concepts (IS-A)	conceptId (string)
GetChildren	Get immediate child concepts	conceptId (string)
GetAncestors	Get all ancestors up to SNOMED root	conceptId (string)
GetDescendants	Get all descendant concepts	conceptId (string), maxResults (int, default 100)
IsA	Check if one concept is a subtype of another	conceptId (string), ancestorId (string)

Configuring Claude Desktop

1. Locate or install the `iris-mcp-server` binary (provided by InterSystems with IRIS AI Hub).
2. Copy the configuration template:

```
cp config/mcp-config.toml ~/snomed-mcp-config.toml
```
3. Edit `~/snomed-mcp-config.toml` — adjust host, port, and credentials if IRIS is not on localhost:

```
[iris.server]
host = "localhost"
port = 1972
username = "CSPSystem"
password = "SYS"
```

4. Add to your Claude Desktop configuration file:

macOS: `~/Library/Application Support/Claude/claude_desktop_config.json` **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

```
{
  "mcpServers": {
    "snomed": {
      "command": "/path/to/iris-mcp-server",
      "args": ["--config=/path/to/snomed-mcp-config.toml", "run"]
    }
  }
}
```

5. Restart Claude Desktop. The SNOMED tools appear in the tool picker (hammer icon).

Configuring Claude Code

Add the same server definition to `~/.claude/settings.json`:

```
{
  "mcpServers": {
    "snomed": {
      "command": "/path/to/iris-mcp-server",
      "args": ["--config=/path/to/snomed-mcp-config.toml", "run"]
    }
  }
}
```

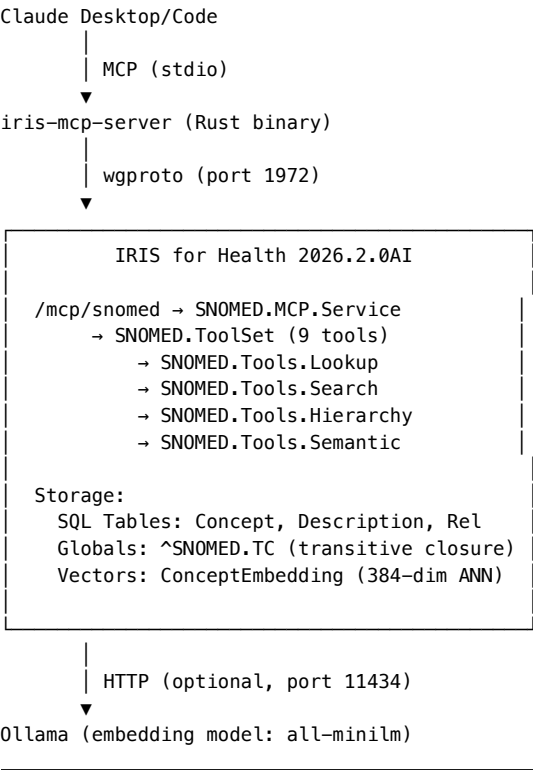
Example Conversations

Once connected, you can ask Claude naturally:

- “What is SNOMED concept 73211009?”
- “Find all concepts related to myocardial infarction”
- “Is atrial fibrillation a type of cardiac arrhythmia?”
- “What are the subtypes of diabetes mellitus?”
- “Find concepts semantically similar to ‘pain in the lower back radiating to the leg’”
- “Show me the parents of concept 22298006”
- “How many descendants does ‘Clinical finding’ have?”

Claude will automatically select and call the appropriate SNOMED tool based on your question.

Architecture Overview



Ports and Services

Port	Protocol	Service
1972	wgproto (binary)	IRIS SuperServer — MCP tool calls route through here
52773	HTTP	IRIS Management Portal and REST endpoints
11434	HTTP	Ollama API — embedding generation for semantic search

Troubleshooting

“Cannot connect to Ollama” during semantic search

Semantic search requires Ollama running with an embedding model. If you only need lexical search and hierarchy navigation, Ollama is not required.

```
# Check Ollama is running
docker compose ps ollama
```

```
# Pull the model if not yet downloaded
docker exec snomed-iris-ollama-1 ollama pull all-minilm
```

MCP tools not appearing in Claude

1. Verify IRIS is running: `docker compose ps`
2. Check the web app exists: open
`http://localhost:52773/csp/sys/sec/%25CSP.UI.Portal.Applications.WebList.zen` and look for `/mcp/snomed`
3. Verify `config.toml` points to the correct host/port
4. Check `iris-mcp-snomed.log` for connection errors

RF2 loading errors

- Ensure you're loading **Snapshot** files (not Full or Delta)
 - File paths must be absolute inside the container
 - Load in order: Concepts → Descriptions → Relationships → TransitiveClosure
-

License Notice

SNOMED CT® is a registered trademark of SNOMED International. This software does not include any SNOMED CT content. You must obtain an appropriate license to use SNOMED CT data in your jurisdiction.